

План створення проєкту EasyStock

Детальний покроковий план: від ідеї до MVP та захисту

Проєкт: EasyStock

Дисципліна: Проектний практикум

Група: ЗІПЗс

Студент: Бондаренко Сергій

Дата: 01.10.2026

1. Короткий опис проєкту

EasyStock - вебсистема для обліку товарів і складських операцій у невеликому магазині або складському приміщенні. Система об'єднує товари, категорії, постачальників, залишки, надходження, списання та історію операцій.

Головна цінність

Працівник бачить актуальний залишок і проводить складську операцію в одному місці, а система автоматично контролює правила та зберігає історію.

2. Наскрізний сценарій MVP

Комірник авторизується, знаходить товар, створює надходження, вводить кількість, система перевіряє дані, записує операцію, оновлює залишок і показує результат. Другий сценарій - списання із перевіркою, що кількість не перевищує залишок.

Етап	Результат	Що потрібно мати
1. Ініціювання	Проблема та межі	Problem statement, SMART, метрики, MVP
2. Вимоги	Перевірюваний backlog	Інтерв'ю, персони, stories, GWT, NFR, MoSCoW
3. Архітектура	Технічний проєкт	Context, components, sequence, ER, prototype, ADR
4. Реалізація	Працездатний MVP	Backend, API, DB, frontend
5. Тестування	Доказ коректності	Позитивні/негативні тести, regression
6. Захист	Демонстрація цінності	README, звіти, walkthrough, pitch

3. Етап 1 - лабораторна №1: ініціювання

За методичними матеріалами лабораторної №1 потрібно отримати не лише ідею, а погоджений набір артефактів: проблему, користувачів, SMART-мету, метрики, MVP, ролі, командну угоду, репозиторій, дошку задач і змістовний commit.

3.1. Вибір проблеми

На старті порівнюємо щонайменше три альтернативи. EasyStock обираємо через зрозумілу цінність, доступність перевірки, реалістичний MVP і велику кількість чітких бізнес-правил.

Альтернатива	Користь	Перевірка	Реалізованість	Ризик	Рішення
Облік ремонтів (FixFlow)	Висока	Середня	Висока	Середній	Відхилено: дублює EasyStock

Альтернатива	Користь	Перевірка	Реалізованість	Ризик	Рішення
					за складністю, але потребує іншої предметної області
Навчальна платформа (StudyHub)	Висока	Середня	Висока	Середній	Відхилено: близька до університетських систем, менше складських бізнес-правил
Складський облік (EasyStock)	Висока	Висока	Висока	Низький	Обрано: реальний сценарій, чіткі сутності, сильна база для UML та MVP

3.2. Problem statement

Працівники невеликих магазинів і складів часто ведуть облік товарів вручну або в декількох файлах, через що витрачають додатковий час на перевірку залишків і допускають помилки під час надходження та списання. EasyStock об'єднує основні складські операції в одній системі та автоматично підтримує актуальний залишок.

3.3. Користувачі та стейкхолдери

Сторона	Інтерес	Вплив	Залучення
Адміністратор	Керування системою та доступом	Високий	Вимоги, review
Комірник	Швидкі надходження та списання	Високий	Інтерв'ю, walkthrough
Менеджер	Актуальні залишки	Середній	Інтерв'ю, acceptance
Власник магазину	Менше помилок і прозорий облік	Високий	Цілі та метрики
Розробники	Реалізованість рішення	Високий	Архітектура, код, тести

3.4. SMART-мета та метрики

SMART-мета: до завершення навчального проєкту розробити та перевірити MVP вебсистеми EasyStock, яка дозволяє авторизованому працівнику переглядати товари, реєструвати надходження та списання і автоматично оновлювати залишки.

Метрика	Базове значення	Ціль	Як виміряти
Час пошуку товару	ручний/невизначений	<= 30 с	5 повторних сценаріїв

Метрика	Базове значення	Ціль	Як виміряти
Критичні операції MVP	не автоматизовані	>= 90% без критичних помилок	тестовий прогін

3.5. MVP та поза межами

Категорія	Функції
MUST / MVP	Login; список товарів; пошук; створення товару; надходження; списання; автоматичний залишок; історія; logout
SHOULD	редагування товарів; категорії; постачальники; low-stock indicator
COULD	штрихкод; CSV; dark mode; статистика
WON'T now	мобільний застосунок; онлайн-оплата; AI прогнозування; Telegram-бот; інтеграція з поштовими сервісами

3.6. Ролі команди

Роль	Основна відповідальність
Координатор/аналітик	проблема, вимоги, backlog, контроль строків
Архітектор	контекст, компоненти, sequence, ADR
Data owner	ER, PK/FK, constraints
UX owner	Figma, стани, walkthrough
Reviewer	трасування вимог, тестування, узгодженість

Якщо команда з 3-5 осіб

Ролі можна поєднувати, але кожен артефакт повинен мати власника та проходити review іншим учасником.

3.7. Git, репозиторій і дошка

Створити репозиторій EasyStock на GitHub/GitLab. Посилання в звіті потрібно замінити на фактичне перед задачею. Для дошки використовувати колонки Backlog -> To Do -> In

Progress -> Review/Test -> Done. WIP рекомендується обмежити 2-3 активними задачами для невеликої команди.

Перший змістовний commit: docs: add project charter and team agreement

4. Етап 2 - лабораторна №2: вимоги та backlog

Лабораторна №2 переводить проблему в перевірювані user stories. У звіті повинні бути 2 інтерв'ю/змодельовані сесії, 2 персони, 10-15 stories, GWT для 5 stories, 4-6 NFR, MoSCoW, межа MVP та review.

4.1. Підготовка інтерв'ю

Провести/змодельовати дві сесії: з комірником і менеджером. Ставити відкриті питання про реальний процес, труднощі, винятки, дані та критерії успіху.

4.2. User stories

ID	Story	Роль	Цінність	MoSCoW	MVP
US-01	Увійти в систему	Працівник	авторизований доступ	Must	Так
US-02	Перегляд товарів	Менеджер	знайти потрібний товар	Must	Так
US-03	Пошук товару	Менеджер	швидко перевірити наявність	Must	Так
US-04	Створити товар	Адміністратор	додати нову позицію	Must	Так
US-05	Редагувати товар	Адміністратор	підтримувати актуальні дані	Should	Ні
US-06	Створити категорію	Адміністратор	класифікувати товари	Should	Так
US-07	Додати постачальника	Адміністратор	вести джерела поставок	Should	Так
US-08	Прийняти поставку	Комірник	збільшити залишок без ручного перерахунку	Must	Так
US-09	Списати товар	Комірник	зменшити залишок із перевіркою доступної кількості	Must	Так
US-10	Автоматично оновити залишок	Система/комірник	бачити актуальні дані після операції	Must	Так
US-11	Переглянути	Комірник	контролювати хто	Must	Так

ID	Story	Роль	Цінність	MoSCoW	MVP
	історію		і що змінював		
US-12	Побачити низький залишок	Менеджер	вчасно поповнити запаси	Should	Hi
US-13	Керувати користувачами	Адміністратор	контролювати ролі та доступ	Should	Hi
US-14	Вийти із системи	Користувач	завершити сеанс	Must	Так

4.3. Acceptance Criteria

Story	Given	When	Then
US-01	Користувач має облікові дані	Вводить правильні дані та натискає «Увійти»	Створено сеанс і відкрито Dashboard
US-03	Відкрито список товарів	Вводить назву/SKU	Показуються лише відповідні товари
US-08	Товар існує	Комірник вводить кількість > 0 і підтверджує	Створено операцію і залишок збільшено
US-09	Залишок = 37	Вводить списання 40	Операція відхиляється, залишок залишається 37
US-11	Є записані операції	Користувач відкриває історію	Бачить дату, товар, тип, кількість і автора

4.4. Нефункціональні вимоги

ID	Тип	Вимога	Перевірка
NFR-01	Продуктивність	95% запитів до списку товарів у локальному середовищі - не більше 2 с при 500 товарах.	Перевірка навантаженням/таймінгами
NFR-02	Цілісність	Заборонити списання кількості, більшої за поточний залишок.	Негативний тест + транзакція
NFR-03	Безпека	Паролі зберігати лише як криптографічні хеші; доступ до API - за ролями.	Перевірка БД + API tests
NFR-04	Сумісність	Основний сценарій повинен працювати у двох сучасних браузерах.	Chrome + Firefox
NFR-05	UX-помилки	Помилка операції не повинна очищати коректно введені поля форми.	UI test
NFR-06	Аудит	Кожна зміна залишку має	Перевірка stock_movements

ID	Тип	Вимога	Перевірка
		містити автора, час, тип і кількість операції.	

4.5. MoSCoW та залежності

MUST формує цілісний сценарій: авторизація -> пошук/вибір товару -> складська операція -> перевірка -> оновлення -> результат. SHOULD та COULD не повинні блокувати цей сценарій.

4.6. Review backlog

Було	Зауваження	Стало
«Система швидка»	Немає метрики	NFR-01: 95% списку <= 2 с при 500 товарах
«Списати товар»	Не визначено обмеження	Додано перевірку quantity <= stock
«Переглянути товари»	Незрозуміло, як знайти	Додано пошук за назвою та SKU
«Історія»	Немає полів аудиту	Додано автор, час, тип і кількість

5. Етап 3 - лабораторна №3: архітектура, дані та інтерфейс

Лабораторна №3 повинна дати узгоджений технічний проєкт: контекст, 3-7 компонентів, критичний sequence/flow, ER-модель, прототип 5-8 екранів, walkthrough та два ADR.

EasyStock - контекстна схема

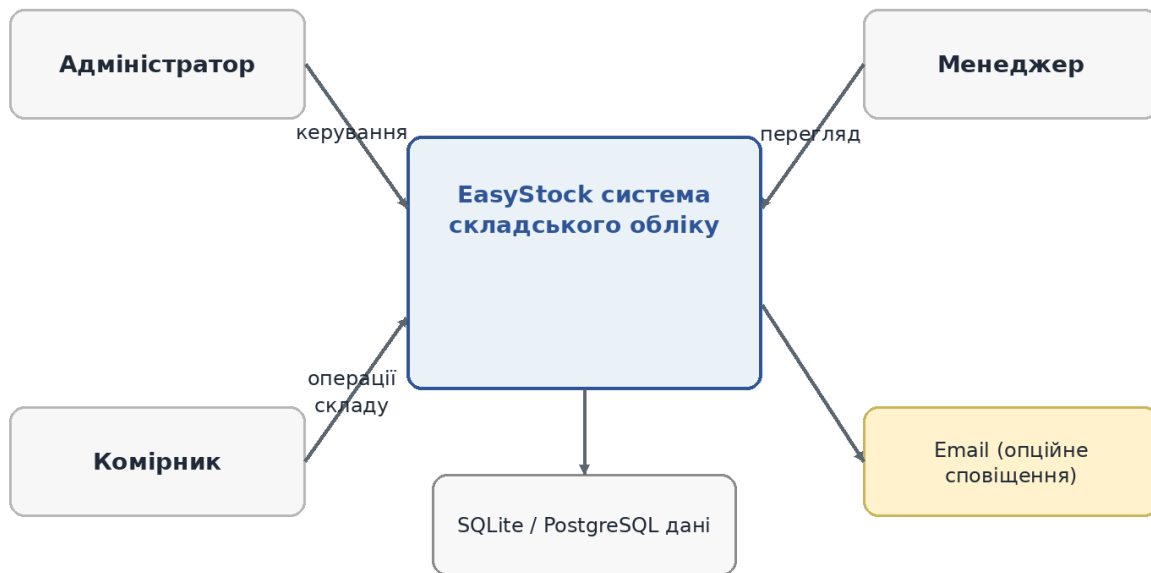


Рисунок 1 - Контекст EasyStock

EasyStock - компонентна архітектура

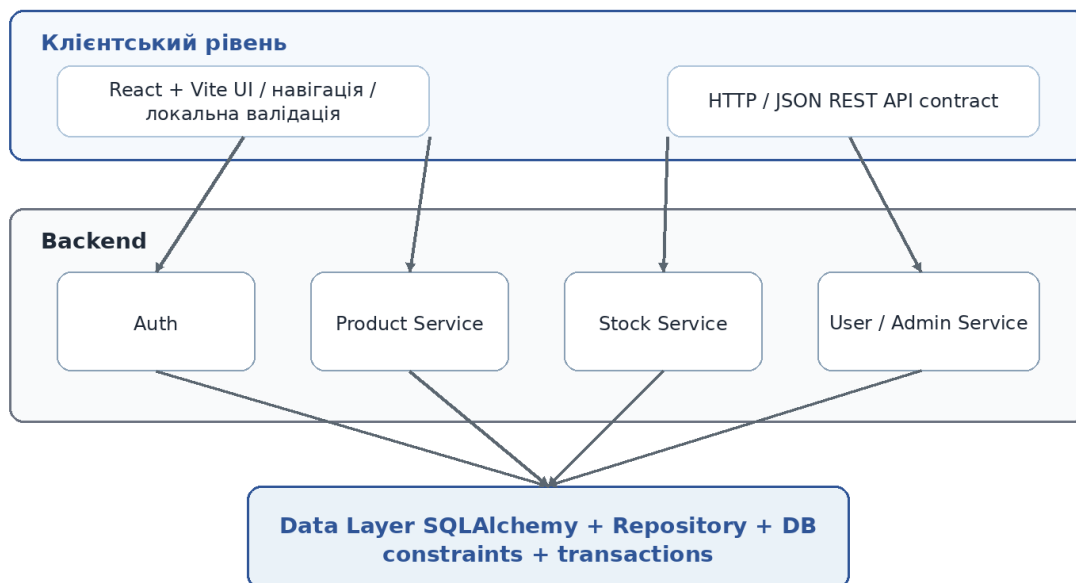


Рисунок 2 - Компонентна архітектура EasyStock

EasyStock - sequence diagram: надходження товару

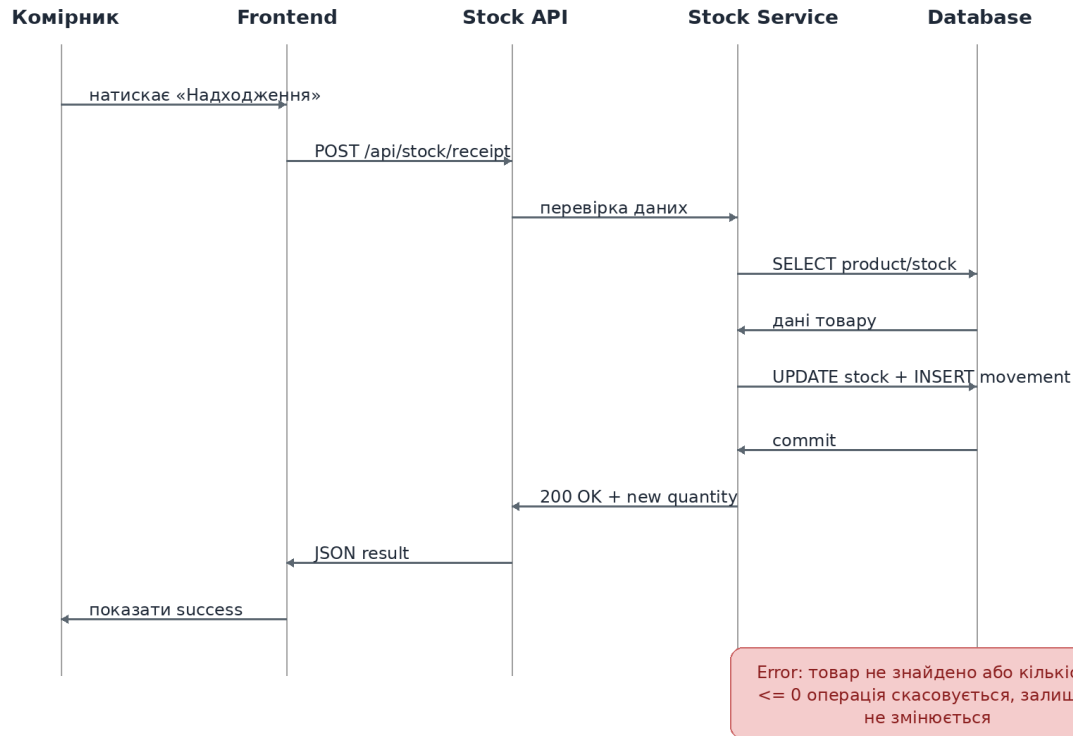
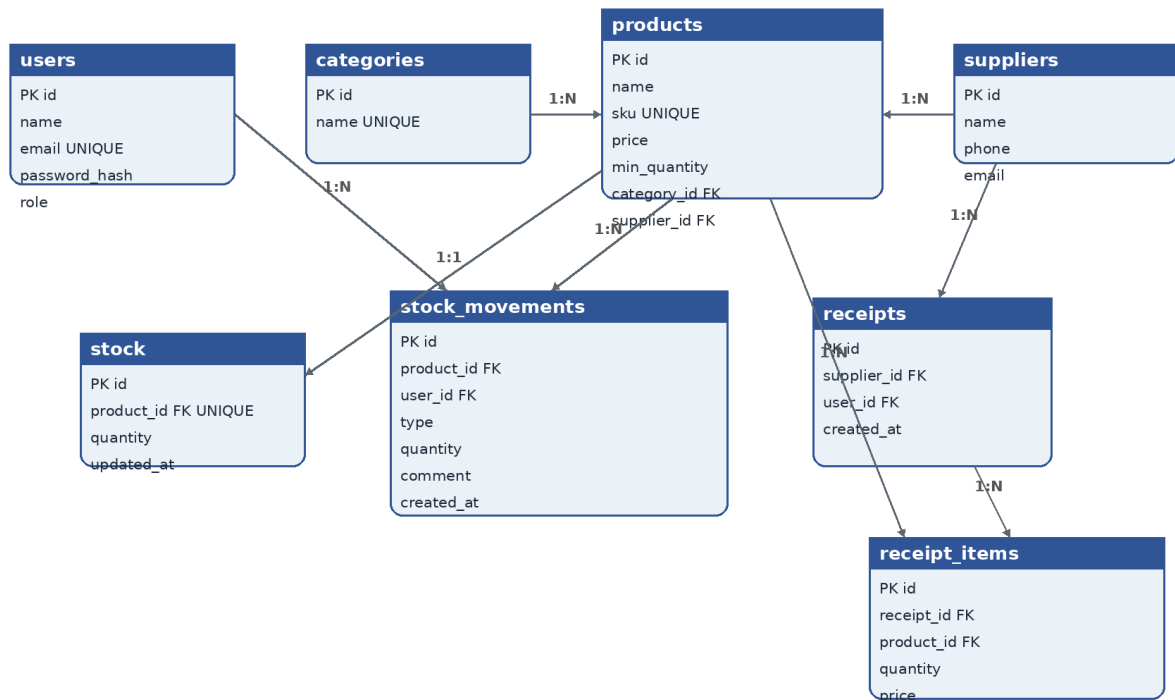


Рисунок 3 - Критичний сценарій надходження

EasyStock - ER-модель



6. Технологічний стек

Шар	Технологія	Причина
Frontend	React + Vite	швидкий SPA, компонентність, проста збірка
Backend	Python + Flask	легкий REST API та невеликий MVP
ORM	SQLAlchemy	зв'язує Python-моделі з БД
DB	SQLite для MVP	мінімум конфігурації; легко переносити
Auth	JWT/secure session	рольовий доступ до API
UML	diagrams.net / PlantUML	редаговані діаграми
Prototype	Figma	клікабельні екрани та walkthrough
API testing	Postman/Insomnia	швидка перевірка API

7. Структура репозиторію

Рекомендована структура:

EasyStock/

├─ README.md

├─ docs/

| ├─ Lab1/

| ├─ Lab2/

| ├─ Lab3/

| └─ adr/

├─ backend/

├─ frontend/

├─ database/

└─ tests/

8. Порядок розробки коду

1. Створити backend та конфігурацію.

- Створити моделі User, Category, Product, Supplier, Stock, StockMovement, Receipt, ReceiptItem.
- Реалізувати авторизацію і рольовий контроль.
- Реалізувати CRUD для товарів та категорій.
- Реалізувати POST /api/stock/receipt та POST /api/stock/writeoff.
- Зробити транзакційну зміну Stock і запис у StockMovement.
- Реалізувати React-сторінки Login, Dashboard, Products, Product Details, Receipt, Write-off, History, Users.
- Додати success/error/empty states.
- Покрити ключові правила тестами.
- Пройти демонстраційний сценарій від login до історії.

9. Головне бізнес-правило

Не можна списати більше, ніж є на складі

При списанні система повинна перевірити `quantity <= current_stock` до зміни залишку. У разі помилки операція скасовується, залишок не змінюється, користувач отримує зрозуміле повідомлення.

10. Тест-план

ID	Перевірка	Очікуваний результат
TC-01	Login з правильними даними	Dashboard відкривається
TC-02	Login з неправильним паролем	Помилка, сеанс не створено
TC-03	Надходження +20 при stock=5	Новий stock=25
TC-04	Списання 3 при stock=25	Новий stock=22
TC-05	Списання 30 при stock=22	Помилка, stock=22
TC-06	quantity=0	Помилка валідації
TC-07	Перевірка History	Є автор, час, тип, кількість
TC-08	Низький залишок $\leq$ min	UI показує попередження

11. 16-кроковий roadmap

Крок	Результат
1	Вибрати тему та сформулювати проблему
2	Описати цільову групу та стейкхолдерів
3	SMART-мета і дві метрики

Крок	Результат
4	Визначити MVP та out of scope
5	Створити GitHub/GitLab та Board
6	Оформити статут і командну угоду
7	Провести 2 інтерв'ю / змодельовані сесії
8	Створити 2 персони та 14 User Stories
9	Додати 5 GWT та 6 NFR
10	Провести MoSCoW і review backlog
11	Побудувати context/components/sequence/ER
12	Створити Figma з 8 екранів
13	Оформити ADR-001 та ADR-002
14	Реалізувати backend/API/DB
15	Реалізувати frontend та інтеграцію
16	Протестувати, оновити README і підготувати захист

12. Що має бути готове перед задачею

- 4 окремі PDF: цей план + Lab1 + Lab2 + Lab3.
- Актуальний backlog та Git-історія.
- Редаговані діаграми у diagrams.net/PlantUML.
- Клікабельний Figma prototype.
- Тестові докази для ключових acceptance criteria.
- Фактичні посилання на репозиторій і дошку замість placeholder-ів.

Важливо

У цьому пакеті підготовлено зміст звітів та навчальні артефакти. Фактичні GitHub-посилання, склад команди та реальний walkthrough іншою людиною потрібно заповнити/виконати перед фінальною задачею.